

CRUD Operations Using Mongoose

Step 1: Install Mongoose

```
npm install mongoose
```

Step 2: Connect with MongoDB Atlas

```
const mongoose = require("mongoose");

mongoose.connect("mongodb+srv://username:password@cluster.mongodb.net/studentDB")
  .then(() => {
    console.log("Database Connected");
  })
  .catch((err) => {
    console.log(err);
  });
```

Step 3: Create Schema

```
const studentSchema = new mongoose.Schema({
  name: String,
  age: Number,
  course: String,
  email: String
});
```

Step 4: Create Model

```
const Student = mongoose.model("Student", studentSchema);
```

Mongoose automatically creates a collection named:

students

CREATE OPERATIONS

Q1. How do we insert one document?

```
const student = new Student({
  name: "Shahid",
  age: 21,
  course: "MERN",
  email: "shahid@gmail.com"
});

await student.save();
```

Q2. How do we insert multiple documents?

```
await Student.insertMany([
  {
    name: "Ali",
    age: 22,
    course: "Java"
  },
  {
    name: "Rahul",
    age: 23,
    course: "Python"
  }
]);
```

READ OPERATIONS

Q3. How do we fetch all documents?

```
const students = await Student.find();  
  
console.log(students);
```

Q4. How do we fetch a single document?

```
const student = await Student.findOne({  
  name: "Shahid"  
});  
  
console.log(student);
```

Q5. How do we find a document by ID?

```
const student = await Student.findById(  
  "6656c1a23f8c9d123456789"  
);  
  
console.log(student);
```

Q6. How do we find students by name?

```
const students = await Student.find({  
  name: "Shahid"  
});
```

Q7. How do we find students by age?

```
const students = await Student.find({  
  age: 21  
});
```

Q8. How do we find students by course?

```
const students = await Student.find({  
  course: "MERN"  
});
```

Q9. How do we find students by email?

```
const students = await Student.find({  
  email: "shahid@gmail.com"  
});
```

UPDATE OPERATIONS

Q10. How do we update one document?

```
await Student.updateOne(  
  {  
    name: "Shahid"  
  },  
  {  
    age: 25  
  }  
);
```

Q11. How do we update multiple documents?

```
await Student.updateMany(  
  {  
    course: "MERN"  
  },  
  {  
    course: "Full Stack"  
  }  
);
```

Q12. How do we update a document by ID?

```
await Student.findByIdAndUpdate(  
  "6656c1a23f8c9d123456789",  
  {  
    age: 24  
  }  
);
```

DELETE OPERATIONS

Q13. How do we delete one document?

```
await Student.deleteOne({  
  name: "Shahid"  
});
```

Q14. How do we delete multiple documents?

```
await Student.deleteMany({  
  course: "Python"  
});
```

Q15. How do we delete a document by ID?

```
await Student.findByIdAndDelete(  
  "6656c1a23f8c9d123456789"  
);
```

COUNT OPERATIONS

Q16. How do we count total documents?

```
const totalStudents =  
await Student.countDocuments();
```

QUICK REVISION

Create

```
save()  
insertMany()
```

Read

```
find()  
findOne()  
findById()
```

Update

```
updateOne()  
updateMany()  
findByIdAndUpdate()
```

Delete

```
deleteOne()  
deleteMany()  
findByIdAndDelete()
```

Count

```
countDocuments()
```